

Quiz Master - V1 Project Report

Name: Sundram Kumar Ray
Roll Number: 23f2002880
Email: 23f2002880@ds.study.iitm.ac.in

Project Details: Problem Statement

Quiz Master - V1 is a multi-user application designed as an exam preparation site for multiple courses. It allows an admin to manage quizzes, subjects, and users, while registered users can attempt quizzes and track their scores.

Approach:

The project was developed using Flask as the backend framework, Jinja2 for templating, and Bootstrap for styling. SQLite was used as the database, ensuring lightweight data storage and easy retrieval. The project follows an MVC (Model-View-Controller) architecture to maintain modularity and scalability.

Novel Features Compared to a Standard Quiz App

1. Modular JavaScript Approach

- **Separation of Concerns:** The frontend JS is structured to keep logic separate for handling questions, user actions, and API interactions.
- **Dynamic Question Handling:** The script dynamically loads and updates question statuses without refreshing the page.

2. Real-time Status Updates

- **Visual Status Indicators:** Each question is color-coded (Answered, Marked for Review, Not Visited).
- **Instant Status Updates:** Question status updates reflect immediately in the UI and backend.

3. Quiz Window Management

- **Controlled Quiz Experience:** The quiz opens in a separate, full-screen, unclosable window.
- **Prevent Accidental Closure:** Users must either submit or leave the quiz formally.

4. Improved User Interaction

- **Jump to Questions:** Users can directly jump to a specific question while maintaining correct state tracking.
- **Next/Previous Navigation:** Even after jumping, navigation remains intact.
- **Review Mechanism:** Users can flag questions and revisit them before submission.

Frameworks and Libraries Used

- **Flask:** Backend framework to handle routing and logic
- **Jinja2:** Template engine for dynamic content rendering
- **SQLite:** Lightweight relational database for storing quiz and user data
- **Chart.js:** Visualization of user performance
- **Bootstrap & CSS:** Frontend styling and responsiveness
- **Javascript:** It enhances interactivity, manages quiz state, and enables seamless API communication in the quiz web application.

Purpose of These Technologies:

- Flask provides a lightweight and efficient backend solution.
- SQLite ensures easy setup and management of data without external dependencies.
- Jinja2 enables dynamic content rendering for an interactive UI.
- Flask-Login ensures user authentication security.
- Chart.js enhances user experience through graphical representation of quiz performance.

Database Schema Design

Tables and Columns:

- Users Table**
 - id (Primary Key)
 - username (Unique Email)
 - password
 - full_name
 - qualification
 - dob
- Subjects Table**
 - id (Primary Key)
 - name
 - description
- Chapters Table**
 - id (Primary Key)
 - subject_id (Foreign Key)
 - name
 - description
- Quizzes Table**
 - id (Primary Key)
 - chapter_id (Foreign Key)
 - date_of_quiz
 - time_duration
- Questions Table**
 - id (Primary Key)
 - quiz_id (Foreign Key)
 - question_statement
 - option_1, option_2, option_3, option_4
 - correct_option
- Scores Table**
 - id (Primary Key)
 - quiz_id (Foreign Key)
 - user_id (Foreign Key)
 - time_stamp_of_attempt
 - total_score
- Question status**
 - id (Primary Key)
 - user_id (Foreign Key → Users)
 - quiz_id (Foreign Key → Quizzes)
 - question_id (Foreign Key → Questions)
 - status (ENUM: Not Visited, Attempted)

Design Rationale:

The relational structure ensures efficient data retrieval and integrity. Foreign key relationships link subjects, chapters, and quizzes while maintaining modularity.

API Design

The backend follows a **RESTful API design** using Flask, ensuring clean separation of concerns and modularity.

1. Authentication APIs

- **User Login:** /api/auth/login (*POST*)
 - **Request:** { "username": "test", "password": "securepass" }
 - **Response:** { "token": "JWT_TOKEN" }
 - **Description:** Authenticates the user and returns a JWT token for session management.
- **User Registration:** /api/auth/register (*POST*)
 - **Request:** { "username": "test", "email": "test@example.com", "password": "securepass" }
 - **Response:** { "message": "User registered successfully" }
 - **Description:** Creates a new user account.

2. Quiz Management APIs

- **Get Quiz Questions:** /api/quiz/<quiz_id>/questions (*GET*)
 - **Response:** { "questions": [{ "id": 1, "text": "What is 2+2?", "options": ["3", "4", "5"] }] }
 - **Description:** Fetches all questions for a specific quiz.
- **Submit Answer:** /api/quiz/<quiz_id>/submit (*POST*)
 - **Request:** { "question_id": 1, "selected_option": 2 }
 - **Response:** { "correct": true, "score": 5 }
 - **Description:** Submits an answer and receives a score update.
- **Mark Question for Review:** /api/question_status/update/<quiz_id> (*POST*)
 - **Request:** { "question_id": 5, "status": "Marked for Review" }
 - **Response:** { "message": "Status updated successfully" }
 - **Description:** Allows users to mark questions for later review.

(The YAML file for API definition is provided in the code file.)

Architecture and Features

Project Organization:

- app.py: Main Flask application file
- models.py: Defines database models
- routes.py: Handles URL routing and logic
- templates/: Contains HTML and Jinja2 templates
- static/: Stores CSS and JS files
- database.db: SQLite database file
- README.md: Instructions on running the application

Implemented Features:

1. **User Management:** Users can register, log in, and take quizzes with secure session handling.
2. **Admin Panel:** Admin has full control over subjects, quizzes, and users, ensuring smooth management.
3. **Quiz System:** Users can attempt quizzes with multiple-choice questions (MCQs) and view their scores in real time.
4. **Analytics Dashboard:** Displays summary charts for quiz performance using Chart.js, offering insightful data visualization.
5. **API Integration:** APIs support CRUD operations on quizzes, subjects, and scores for seamless data management.

Video Link: <https://drive.google.com/file/d/1QBSIDvdiTxXJxgt9DWg-HzHxHDCOKJAG/view?usp=sharing>

Conclusion

This quiz app is **not just a basic multiple-choice system** but an **interactive, modular, and user-friendly** system. The combination of structured API design, efficient front-end handling, and real-time tracking makes it stand out in terms of usability and security.